SIMO Buck Converter

EE 396: Design Lab

Shantanu Barai: 230108011

Aryan Kumar: 230108010

Supervisor: Dr. Shabari Nath

Department of Electronics and Electrical Engineering

Indian Institute of Technology, Guwahati

Guwahati, 781039

Contents

- **Introduction**

- **Circuits**
  - Power Circuit
  - Driver Circuit
  - Theoretical analysis

- **Gate Pulse Generation using FPGA**
  - Board used
  - Pulse Generation
    - Logic
    - Code

- **PCB Design of our circuit**
  - Schematic
  - Layout

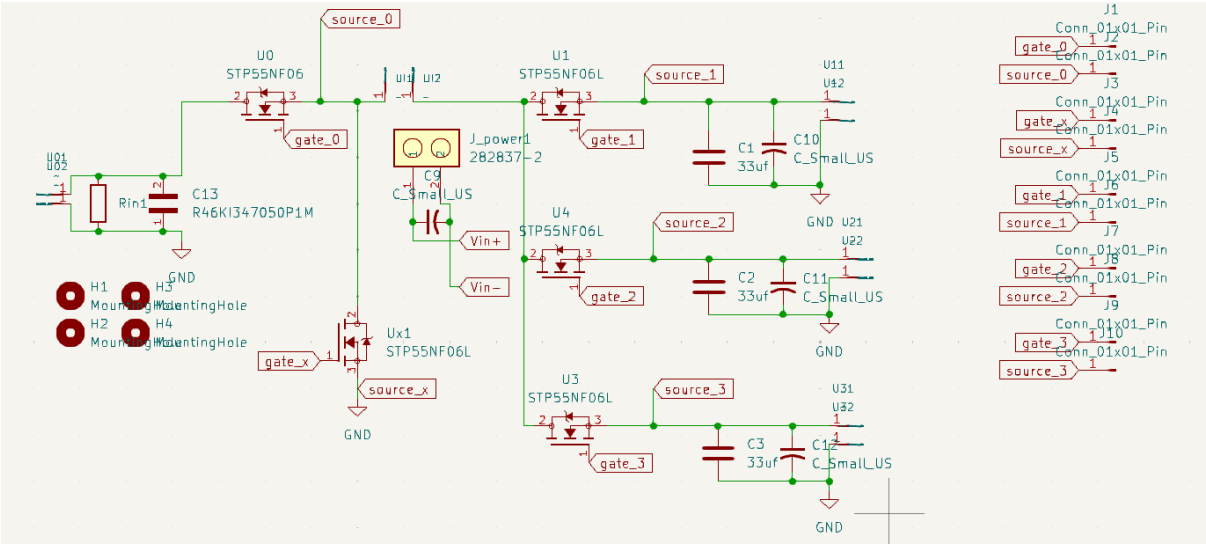
- **Results and Observations**
- **Bill of Materials**
- **Conclusion**
- **Future Work**

Introduction

Modern electronic systems often require multiple voltage levels from a single power source, making efficient power conversion essential. A DC–DC converter is used to convert one DC voltage level to another, and among these, the buck converter is widely used for stepping down voltage. In this project, a **Single-Input Multi-Output (SIMO) buck converter** is designed to generate three regulated output voltages from a single input using controlled MOSFET switching and filtering. This approach reduces component count, size, and overall system complexity while ensuring efficient power distribution. Such converters find applications in microcontroller and FPGA power supplies with multiple voltage rails, portable and battery-operated devices, IoT and communication systems, as well as automotive and other embedded electronic systems.

Circuits

- **Power circuit**



This circuit implements a **Single-Input Multi-Output (SIMO) buck converter** using multiple MOSFET switches. A single DC input is supplied at the left, which is first conditioned and then fed to a common switching node. The main switching MOSFET (U0/Ux1) controls the energy transfer from the input.

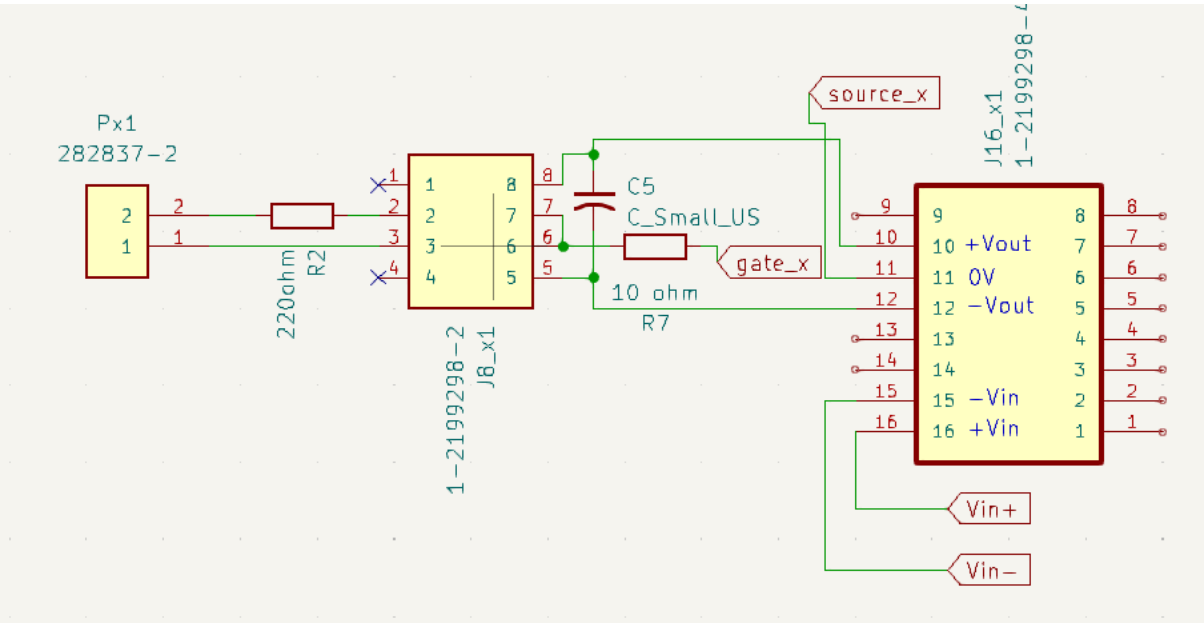
Those 2 Switches have complementary switching by supplying complementary PWMs into there gates using an FPGA.

From this common node, three separate MOSFETs (U1, U4, U3) are used to **time-share the input energy** across three output channels. Each MOSFET is driven by an independent gate signal (gate_1, gate_2, gate_3), allowing selective delivery of power to each output.

At each output, capacitors (C1, C2, C3 along with smaller parallel capacitors) act as **filters**, smoothing the pulsed voltage into stable DC outputs (source_1, source_2, source_3). The smaller SMD capacitors are used for filtering high frequency noises whereas electrolytic capacitors filter out low frequency noises. The connectors on the right provide access to gate signals and output nodes for control and measurement.

Overall, the circuit uses **controlled switching and energy sharing** to efficiently generate multiple regulated DC outputs from a single input source while minimizing component count.

- # Driver circuits



This circuit is an **isolated gate-driving module** where the input signal passes through an optocoupler (via a current-limiting resistor) to provide electrical isolation, and the optocoupler's output, powered by a dedicated isolated DC-DC

converter drives a MOSFET gate through a small resistor (to control switching speed) and a capacitor (to suppress noise and false triggering); by replicating this block, you independently control multiple MOSFETs in your system—one as the main switch, one replacing the diode for synchronous rectification, and the remaining for selecting which output is energized while maintaining isolation, noise immunity, and flexible floating operation.

- # Theoretical Analysis

Vin = 20 V
Vout = 10 V
Duty cycle (D) = 0.5
Inductor (L) = 560 μH = 560×10^{-6} H
Capacitor (C) = 33 μF = 33×10^{-6} F
Switching frequency (f_s) = 20 kHz = 20000 Hz

---

1) Inductor Current Ripple (ΔI_l):

$\Delta I_l = [(Vin - Vout) \times D] / (L \times f_s)$

$\Delta I_l = [(20 - 10) \times 0.5] / (560 \times 10^{-6} \times 20000)$
$\Delta I_l = 5 / 11.2$
$\Delta I_l \approx 0.446$ A

---

2) Output Voltage Ripple (ΔVout):

$\Delta Vout = \Delta I_l / (8 \times f_s \times C)$

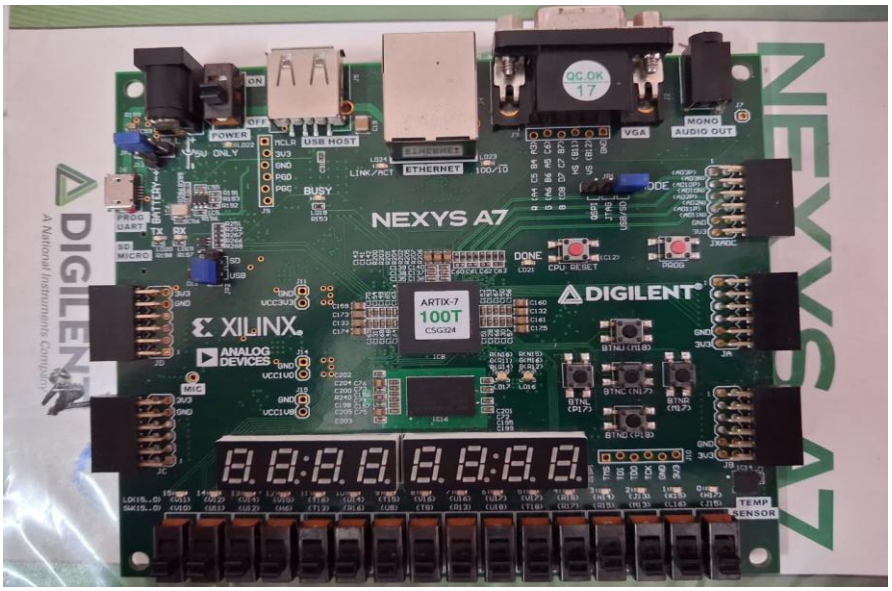
$\Delta Vout = 0.446 / (8 \times 20000 \times 33 \times 10^{-6})$
$\Delta Vout = 0.446 / 5.28$
$\Delta Vout \approx 0.0845$ V

Final Results:
Inductor current ripple ≈ 0.45 A
Output voltage ripple ≈ 84.5 mV

Gate Pulse Generation using FPGA

• FPGA (Nexys A7 100T)

The Nexys A7-100T is an FPGA development board based on the Xilinx Artix-7 (XC7A100T) featuring around 100k logic cells, 240 DSP slices, and ~4.8 Mb of block RAM, making it suitable for complex digital and control applications. It includes a built-in 100 MHz clock oscillator that can be scaled using internal PLL/MMCM blocks to generate required frequencies (like PWM signals), along with 128 MB DDR2 memory and 16 MB flash for storage. The board provides rich I/O such as switches, LEDs, seven-segment displays, UART, VGA, Ethernet, and Pmod connectors, and is commonly used with Vivado for FPGA-based system design.



• Pulse Generation

Logic

This design implements **Buck converter** using an FPGA. Instead of a diode, a second MOSFET is used, making the system more efficient. The FPGA generates PWM signals, ensures safe switching using dead-time, and allows user control through buttons.

A counter running on a 100 MHz clock generates a periodic signal. Based on this, a PWM waveform is created with two selectable duty cycles: **30% and 50%**. The duty cycle determines how long the high-side switch remains ON, thereby controlling the output voltage of the buck converter.

Complementary Switching:

Two outputs (pwm_a and pwm_b) drive two MOSFETs:

pwm_a → high-side switch

pwm_b → low-side switch

They operate in a complementary manner, ensuring that when one is ON, the other is OFF.

Dead Time Insertion between these two Complementary switches:

Dead-time is a short delay inserted between turning OFF one MOSFET and turning ON the other to prevent shoot-through in the synchronous buck converter. In your design, this is implemented using a counter (dead_cnt) inside an FSM, where both pwm_a and pwm_b remain OFF until the counter reaches the set value DEAD_TIME = 200. The duration of this delay depends on the clock period. With a clock frequency of 100 MHz, the clock period is $T = \frac{1}{100\times10^6} = 10$ ns. Therefore, the total dead-time is 200×10 ns $= 2000$ ns $= 2\ \mu s$. This ensures safe switching, though a larger dead-time can slightly reduce efficiency due to diode conduction.

Multioutput Buttons and Debounce Logic:

Three debounced buttons control three switches (sw1, sw2, sw3). Each button press toggles its corresponding output using **edge detection**, ensuring one clean toggle per press.

Mechanical buttons produce noisy signals due to bouncing. The debounce module ensures that a button press is only registered after it remains stable for about **10 ms**, preventing false triggering.

Code

```
module pwm_deadtime_nexysA7_fixedDuty #(
parameter CLK_FREQ=100_000_000,parameter PWM_FREQ=20_000,parameter DEAD_TIME=200)(
input wire clk,input wire rst,
input wire duty_sel,
input wire btn1,input wire btn2,input wire btn3,
output reg pwm_a,output reg pwm_b,
output reg sw1,output reg sw2,output reg sw3);
// ================= PERIOD =================
localparam PERIOD=CLK_FREQ/PWM_FREQ;
localparam DUTY_30=(PERIOD*30)/100;
localparam DUTY_50=(PERIOD*50)/100;

wire[12:0] duty_selected;
assign duty_selected=(duty_sel)?DUTY_50:DUTY_30;
// ================= COUNTER =================
reg[12:0] counter;
always@(posedge clk or posedge rst) begin
if(rst) counter<=0;
else if(counter==PERIOD-1) counter<=0;
else counter<=counter+1;
end
// ================= RAW PWM =================
wire pwm_raw=(counter<duty_selected);
// ================= DEAD-TIME FSM =================
reg[15:0] dead_cnt;
reg state;

always@(posedge clk or posedge rst) begin
if(rst) begin pwm_a<=0;pwm_b<=0;state<=0;dead_cnt<=0;end
else begin
case(state)
```

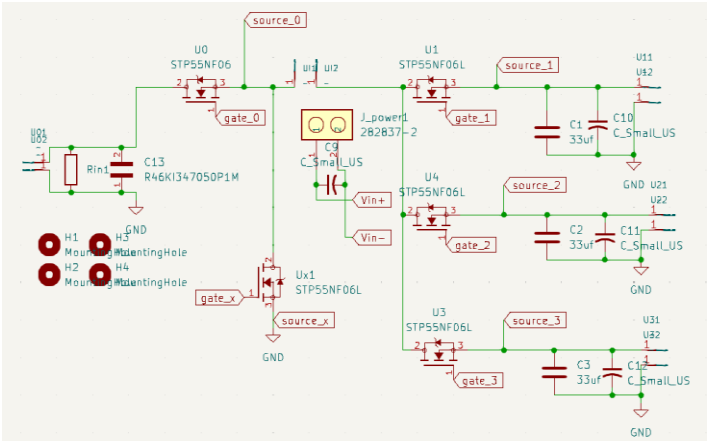
```
0: begin
if(!pwm_raw) begin pwm_a<=0;pwm_b<=0;
if(dead_cnt<DEAD_TIME) dead_cnt<=dead_cnt+1;
else begin dead_cnt<=0;state<=1;end end
else begin pwm_a<=1;pwm_b<=0;dead_cnt<=0;end
end

1: begin
if(pwm_raw) begin pwm_a<=0;pwm_b<=0;
if(dead_cnt<DEAD_TIME) dead_cnt<=dead_cnt+1;
else begin dead_cnt<=0;state<=0;end end
else begin pwm_a<=0;pwm_b<=1;dead_cnt<=0;end
end
endcase end end
// ================== DEBOUNCE MODULE INSTANCES ==================
wire db1,db2,db3;

debounce d1(.clk(clk),.rst(rst),.btn_in(btn1),.btn_out(db1));
debounce d2(.clk(clk),.rst(rst),.btn_in(btn2),.btn_out(db2));
debounce d3(.clk(clk),.rst(rst),.btn_in(btn3),.btn_out(db3));
// ================== TOGGLE LOGIC ==================
reg db1_d,db2_d,db3_d;
always@(posedge clk or posedge rst) begin
if(rst) begin sw1<=0;sw2<=0;sw3<=0;db1_d<=0;db2_d<=0;db3_d<=0;end
else begin
if(db1&~db1_d) sw1<=~sw1;
if(db2&~db2_d) sw2<=~sw2;
if(db3&~db3_d) sw3<=~sw3;
db1_d<=db1;db2_d<=db2;db3_d<=db3;
end end
endmodule
```

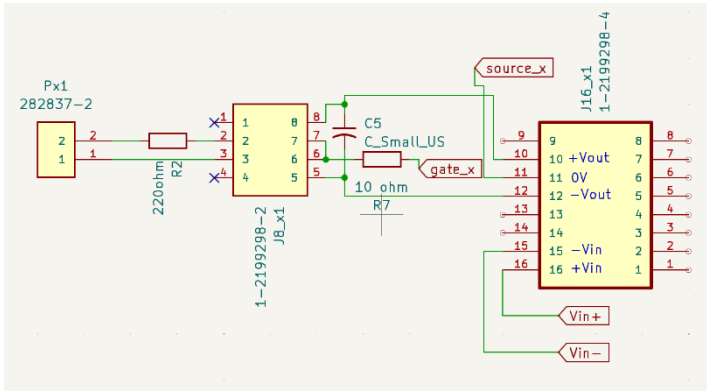
PCB Design of our circuit

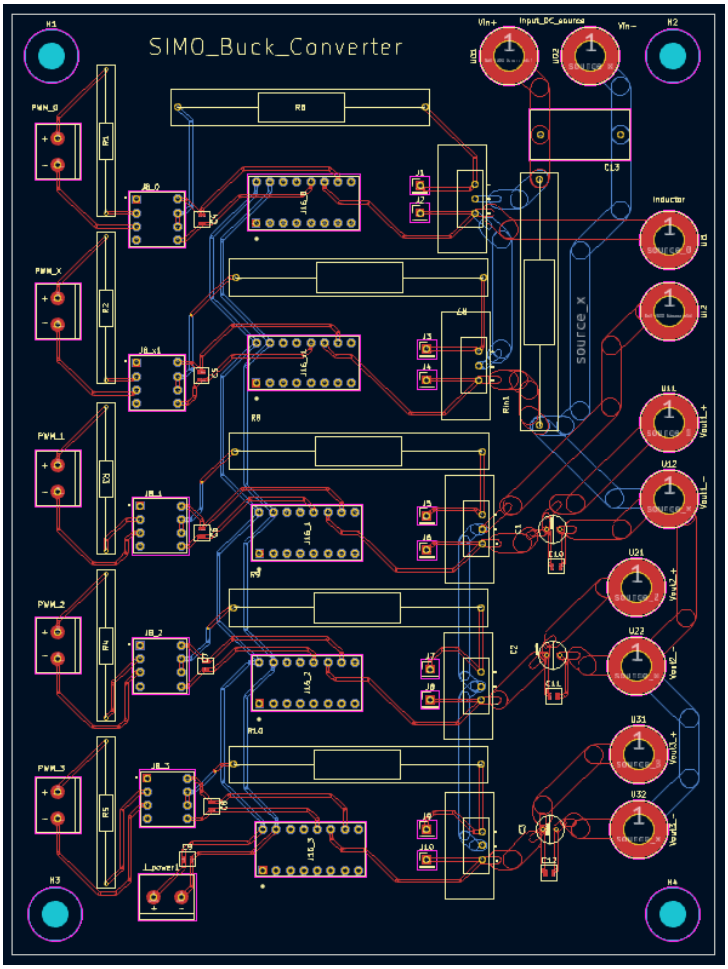
Schematic

Layout

Power Circuit





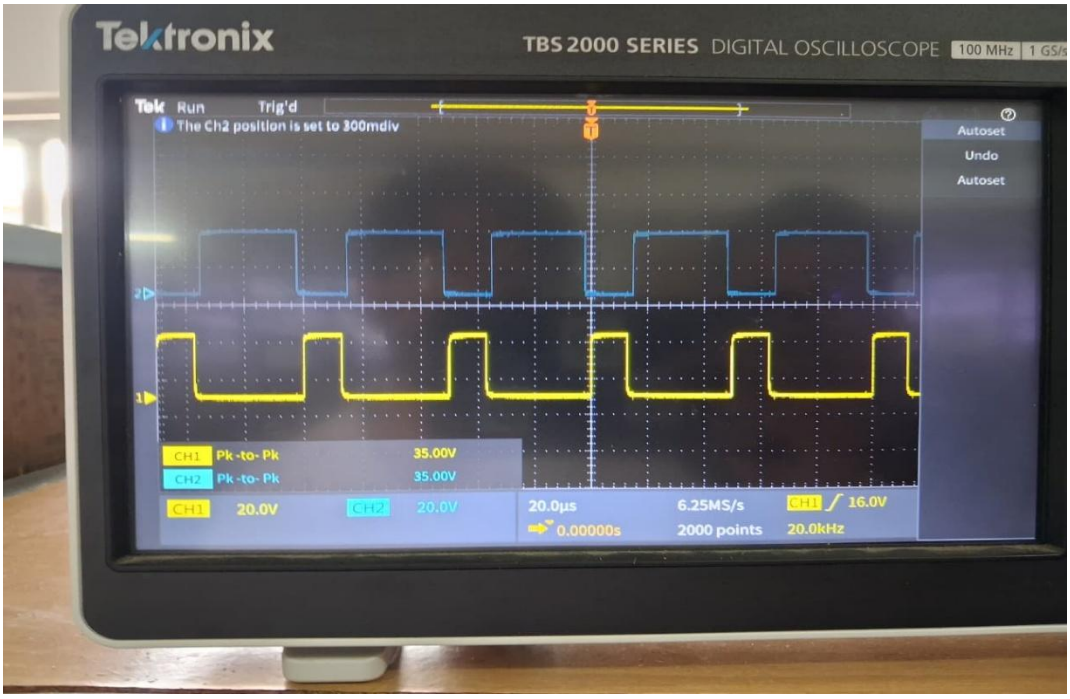


Driver Circuit

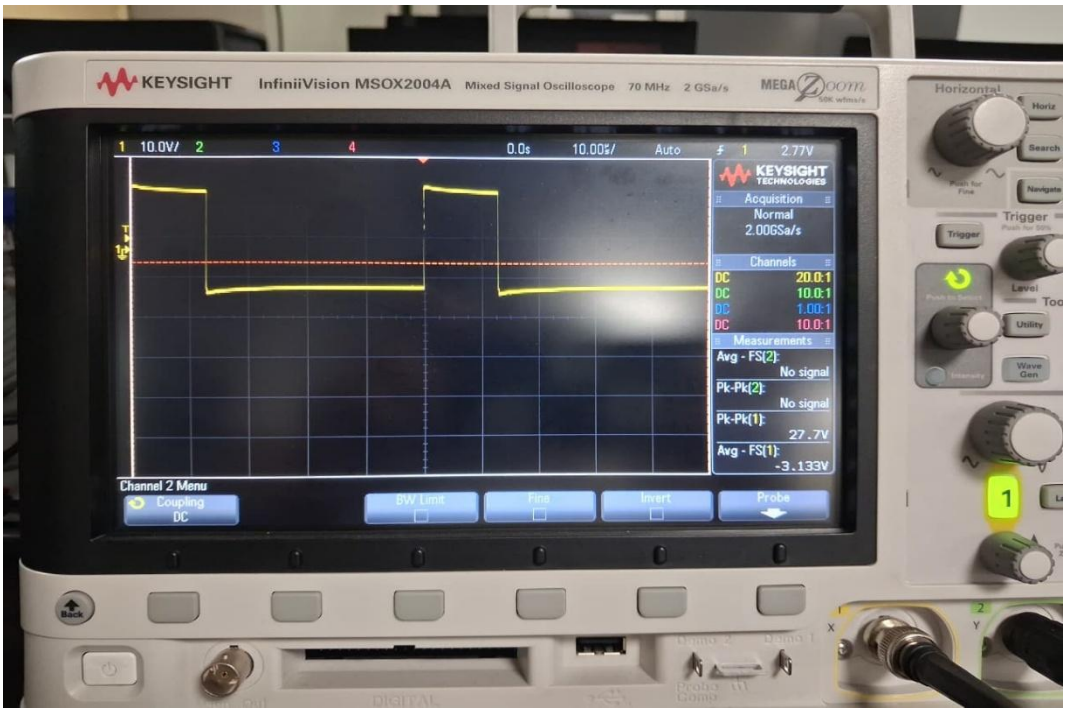
Dimensions of PCB: 13cm x 17.5cm

Results and observations

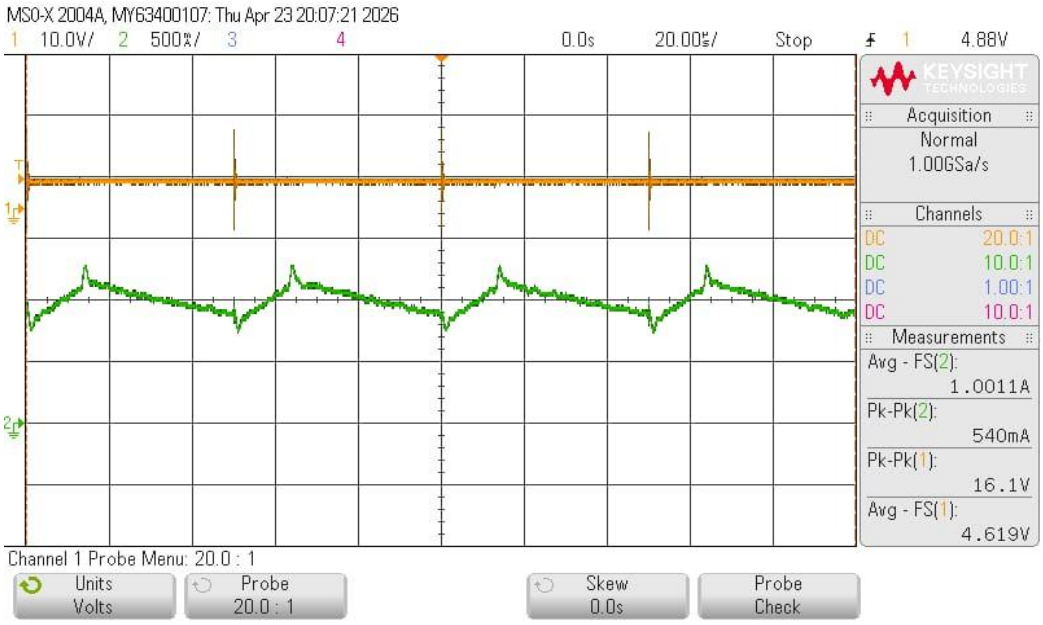
1) PWM generated by FPGA:



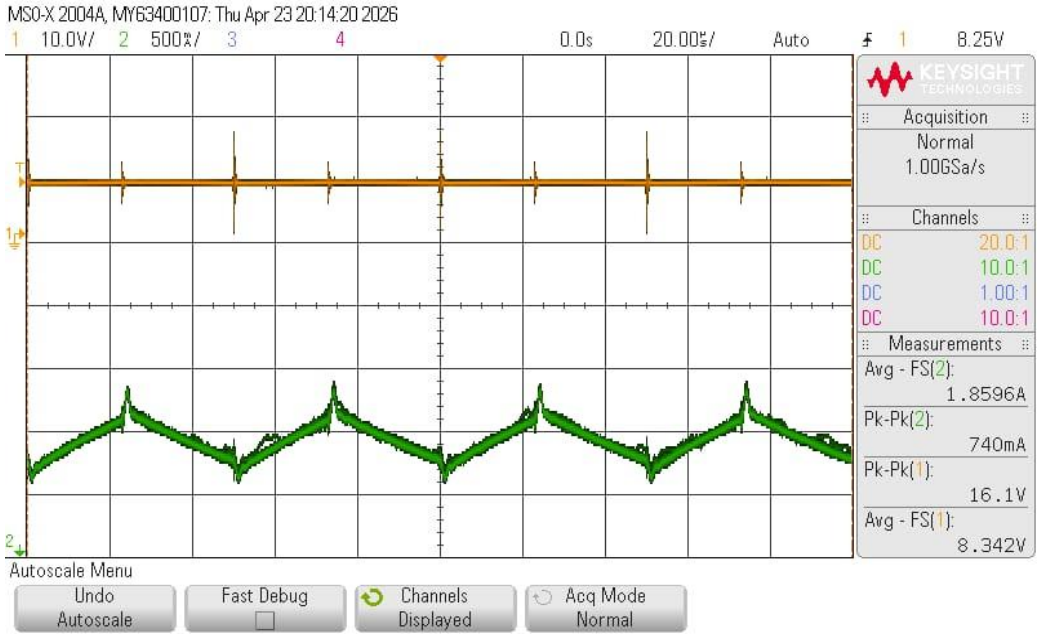
2) PWM output from optocoupler given to Gates:



3) Inductor current and output voltage waveform:



30 % Duty cycle



50 % Duty cycle

Bill of Materials

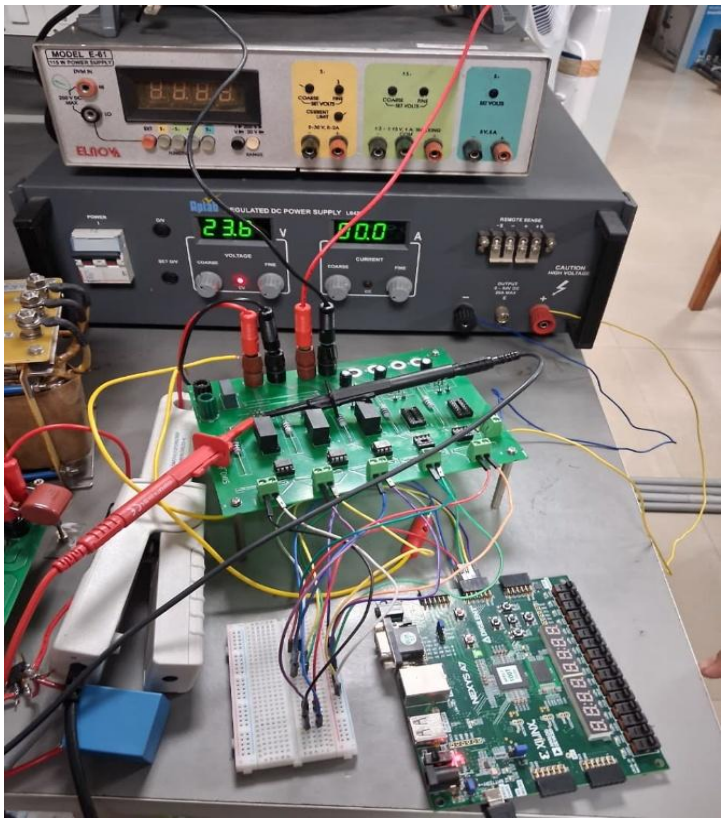
A Bill of Materials (BOM) is an essential component of any hardware design project, providing a detailed list of all components used in the circuit. It includes critical information such as the part name, manufacturer, part number, quantity, value.

Sr no.	Component	Part no.	Specification	Manufacturer	Quantity
1	Terminal Blocks 5mm	MaiXu_MX126-5.0-02P	Pitch = 5mm	MAX Asian Brands	6
2	Optocouplers	HCPL3120	2.5A 3750Vrms 8-DIP	BROADCOM	5
3	Isolated DC-DC converter	MGJ2D241509SC	15V -8.7V 80mA, 40mA 21.6V - 26.4V	MURATA POWER SOLUTIONS	5
4	N channel MOSFETS	STP55NF06	60 V 50A 110W	STMicroelectronics	5
5	Metallized polypropylene Film capacitor	X2 MKP	0.47uF 310VAC	STcapacitors	1
6	MLCC SMD Capacitor	08051C104JAT2A	0.1 uF, 100 V, 0805	KYOCERA AVX	9
7	Electrolytic capacitors	MCKSK050M330D11S	33uF 50V	MULTICORP RPO	3

Results

The buck converter was successfully analyzed for the given parameters, yielding an inductor current ripple of approximately **0.45 A** and an output voltage ripple of about **84.5 mV**, confirming proper operation in continuous conduction mode and validating the chosen component values for stable performance.

Conclusion



The designed buck converter with the given parameters operates effectively, producing the desired output voltage with controlled inductor current ripple and acceptable output voltage ripple. The use of appropriate inductance, capacitance, and switching frequency ensures stable operation in continuous

conduction mode (CCM), while the overall system demonstrates reliable performance for regulated DC power conversion and validates the theoretical design calculations.

Future Work (Closed-Loop Control)

Future improvements can focus on implementing a closed-loop control system to achieve precise output voltage regulation under varying load and input conditions. By incorporating feedback (via voltage sensing) and using a controller such as PID implemented on an FPGA (e.g., Nexys A7), the duty cycle can be dynamically adjusted to minimize steady-state error, reduce ripple further, and improve transient response. Additionally, advanced control techniques and optimized gate driving can enhance efficiency, stability, and overall performance of the converter.